

역등성 보장 requestId 기반 중복 TX 방어

Idempotency · UUID Key · DB-First · At-Least-Once Safety

M3 VASP 추상화

S16

Day 04

오늘 세션 구성

강의 20분 + 실습 35분 · M3 · S16

§ 1 왜 멍등성인가

- At-Least-Once 전달 환경
- 중복 발행 시나리오
- requestId 없을 때의 위험

§ 2 requestId 설계

- UUID as Idempotency Key
- submitMintRequest 흐름
- DB INSERT 먼저 원칙
- VaspMockClient 멍등성
- 핸들러 가드: throw vs return

§ 3 실습

- [1] 정상 흐름
- [2] 멍등성 검증
- [3] VASP 실패 → FAILED
- [4] 핸들러 가드

학습 목표

- › [1] 정상 흐름: REQUESTED → SUBMITTED 전환 및 txHash 저장 확인
- › [2] VASP 멍등성: 같은 requestId 반복 호출 시 동일 txHash 반환 검증
- › [3] VASP 실패 → DB FAILED 레코드 생성 및 failReason 저장 확인
- › [4] 핸들러 가드: 이미 처리된 상태에서 handleMined 재호출 시 throw 없음 검증

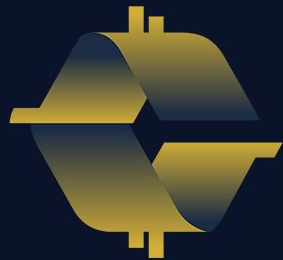
01

왜 멍등성이 필요한가

At-Least-Once 환경에서 중복 요청은 예외가 아닌 기본값이다

§ 1

At-Least-Once



COINCRAFT
ACADEMY

At-Least-Once 전달 환경

§ 1 왜 멍등성인가 · M3 S16

핵심 전제: 메시지 큐는 **정확히-한-번(Exactly-Once)**을 보장하지 않는다. 같은 요청이 두 번 올 수 있다는 것을 설계에 반영해야 한다.

네트워크 타임아웃

요청이 처리되었으나 응답이 손실됨 → 클라이언트가 타임아웃으로 판단 → 동일 요청 재전송 → TX 2개 발행 위험

Consumer 재시작

처리 중 Consumer 프로세스 종료 → 재시작 후 미확인 메시지 재처리 → NFT 2개 민팅 위험 발생

브로커 장애

브로커 failover 과정에서 중복 배달 발생 → 동일 이벤트가 여러 Consumer에게 전달될 수 있음

재시도 없이 처리 → 일시 장애 시 TX 유실 → 안정성 ↓

단순 재시도 → 이미 처리된 요청 재실행 → 중복 발행 ↑

해법: requestId(UUID)를 Idempotency Key로 사용 — VASP는 동일 requestId 재수신 시 신규 TX 없이 기존 결과를 반환한다

중복 발행 시나리오

§ 1 왜 먹등성인가 · M3 S16

시나리오	발생 원인	결과
시나리오 1 네트워크 재시도	응답 손실 → 클라이언트 재전송	TX 2개 발행 → 잔액 2배 차감
시나리오 2 Consumer 재처리	ACK 누락 → 메시지 재배포	NFT 2개 민팅 → 희소성 파괴
시나리오 3 VASP 응답 손실	전송 성공 후 응답 경로 단절	성공/실패 판단 불가 → 상태 불명

결과물 — 비즈니스 임팩트

- › 잘못된 잔액: 동일 출금이 두 번 처리되어 사용자 계정 잔액 불일치
- › 감사 로그 불일치: TX 레코드 2개 / 실제 발행 1회 → 컴플라이언스 문제
- › 고객 민원: "같은 금액이 두 번 빠졌어요" — 운영 비용 증가

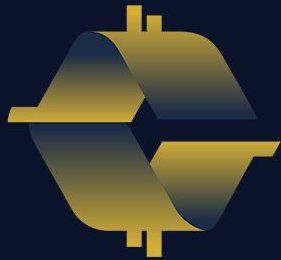
핵심: requestId 없이는 VASP가 중복 요청을 구분할 수 없다 — 동일 내용의 요청 2개는 서로 다른 TX로 처리된다

02 requestId 설계

UUID · DB-First · VaspMockClient · 핸들러 가드

§ 2

Idempotency Key



COINCRAFT
ACADEMY

UUID as Idempotency Key

§ 2 requestId 설계 · M3 S16

requestId 발급 원칙

- › 발급 시점: 요청 생성 시 단 1회 — 재시도 시 재사용
- › 발급 방법: `crypto.randomUUID()` — RFC 4122 v4
- › 전달 범위: TxStateMachineService → VaspTxClient 전 레이어 관통
- › 보관 위치: DB `tx_requests.request_id` UNIQUE 컬럼
- › VASP 역할: 동일 requestId 재수신 → 기존 txHash 반환 (신규 TX 생성 안 함)

VaspMockClient 중복 판별 로직

```
if (this.submitted.has(requestId)) {  
    // 이미 처리된 요청 → 캐시된 txHash 반환  
    return this.submitted.get(requestId);  
}  
  
// 신규 요청 → 새 txHash 생성 후 Map 등록  
const txHash = generateTxHash(requestId);  
this.submitted.set(requestId, txHash);  
return txHash;
```

같은 requestId 3번 → 항상 **동일 txHash** 반환
다른 requestId → **새 txHash** 생성

IdempotencyGuard — 멍등성 영속화

§ 2 requestId 설계 · M3 S16

실제 운영 코드 — IdempotencyGuard

```
// NFTIssuedProcessor.ts
const key = `NFTIssued:${requestId}`;
await idempotency.run(key, async () => { /* 처리 */ });

// IdempotencyGuard.run() 내부
const claimed = await store.tryMark(key, ttl);
if (!claimed) return false; // 중복 → 스킵
await fn();                // 최초 → 실행
```

tryMark() — check + mark를 원자적 단일 연산으로 통합. Race Condition 제거.

저장소 교체 가능 (Strategy 패턴)

InMemoryIdempotencyStore

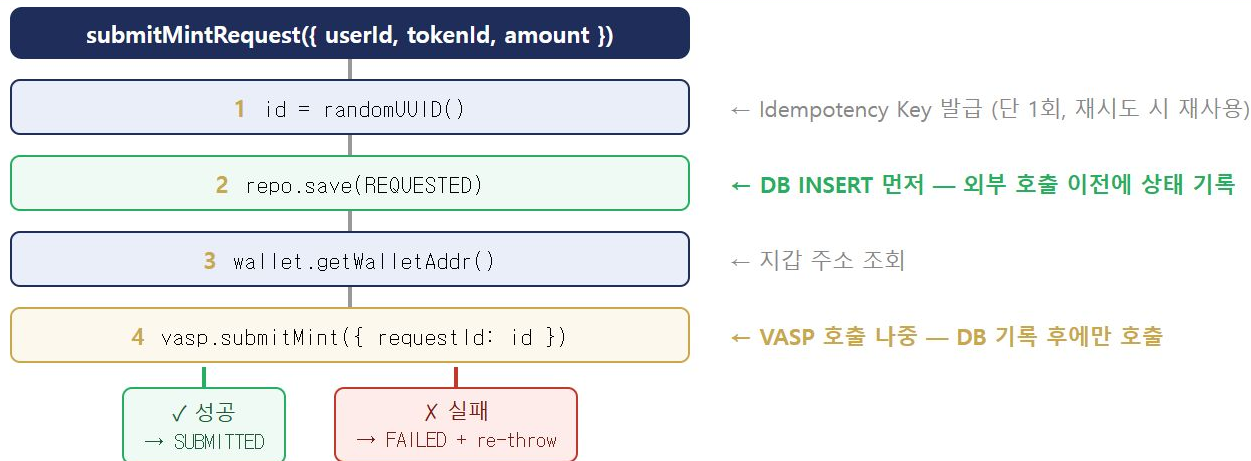
- 개발·테스트용 / Map + TTL
- 재시작 시 초기화 — 운영 부적합

RedisIdempotencyStore (운영)

- SET key NX EX ttl — 단일 명령 원자적
- 재시작 후에도 키 유지 (TTL 7일)
- 수평 확장 시에도 Race Condition 없음

submitMintRequest 흐름

§ 2 requestId 설계 · M3 S16



순서 불변 원칙: 2(DB) → 4(VASP) 순서는 바뀌지 않는다. DB 기록 없이 VASP를 먼저 호출하면 크래시 후 상태 추적 불가.

DB INSERT 먼저 원칙

§ 2 requestId 설계 · M3 S16

✓ DB 먼저 (올바름)

- DB에 REQUESTED 레코드 생성
- VASP 호출 — 성공 시 SUBMITTED 전환
- 크래시 발생 시 → DB에 REQUESTED 기록 남음
- 재시작 후 requestId로 상태 확인 가능
- 중복 방지: 같은 requestId 재처리 → 기존 레코드 반환

X VASP 먼저 (위험)

- VASP에 TX 전송 성공
- DB INSERT 직전 크래시 발생
- DB 기록 없음 → TX는 블록체인에 존재
- 재시작 후 상태 불명 → 중복 발행 시도
- requestId 없으면 VASP도 구분 불가

역등성의 출발점: "상태를 먼저 기록하고, 외부로 나중에 호출한다" — DB는 진실의 원천(Source of Truth). VASP 응답이 없어도 DB 기록으로 상태 복구 가능.

VaspMockClient 멍등성 구현

§ 2 requestId 설계 · M3 S16

```
class VaspMockClient {  
  private submitted = new Map<string, string>();  
  private counter = 0;  
  
  async submitMint(req: {  
    requestId: string;  
    walletAddr: string;  
    tokenId: string;  
    amount: number;  
  }): Promise<string> {  
    const { requestId } = req;  
    if (this.submitted.has(requestId)) {  
      return this.submitted.get(requestId)!;  
    }  
    const n = String(this.counter++).padStart(4, '0');  
    const txHash = `0xMOCK_${n}_${requestId.slice(0,8)}`;  
    this.submitted.set(requestId, txHash);  
    return txHash;  
  }  
}
```

동작 검증

호출	requestId	결과
1회차	uuid-A	0xMOCK_0000_uuid-A
2회차 (중복)	uuid-A	동일 txHash 반환
3회차 (중복)	uuid-A	동일 txHash 반환
신규 요청	uuid-B	0xMOCK_0001_uuid-B

counter는 테스트 편의용 — txHash에 순번을 붙여 몇 번째 신규 TX인지
눈으로 파악 가능

핸들러 가드: throw가 아닌 return

§ 2 requestId 설계 · M3 S16

구현 코드

```
async handleMined(
  txId: string,
  blockNumber: number
): Promise<void> {
  const req = await this.repo.findById(txId);

  // 가드: 이미 처리된 상태면 조용히 무시
  if (req.status !== 'PENDING'
    && req.status !== 'SUBMITTED') {
    return; // throw 하지 않음!
  }

  await this._transition(
    req, 'CONFIRMED', { blockNumber }
  );
}
```

왜 throw가 아닌가

throw를 사용하면

- 메시지 처리 실패로 간주
- DLQ(Dead Letter Queue)로 이동
- 운영팀 수동 처리 필요 → 운영 부담 증가
- 알람 발생 → 불필요한 인시던트 생성

return을 사용하면 (올바름)

- 조용히 무시 (no-op)
- 큐가 다음 메시지로 진행
- DLQ 이동 없음 → 자동 처리

원칙: 중복은 오류가 아니라 **예상된 케이스**다 — at-least-once 환경에서 이미 처리된 메시지 재배달은 정상 동작이다

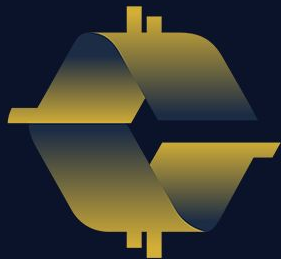
03

실습

5개 테스트로 멍등성 원칙 전체를 직접 검증한다

§ 3

`npm run exercise:s16:answer`



COINCRAFT
ACADEMY

실습 [1] — 정상 흐름

§ 3 실습 · M3 S16

테스트 코드

```
const result = await svc
  .submitMintRequest({
    userId: 'user-001',
    tokenId: 'TOKEN-A',
    amount: 100
  });

const saved = await repo
  .findById(result.id);

expect(saved.status)
  .toBe('SUBMITTED');
expect(saved.txHash)
  .toBeTruthy();
```

검증 항목

항목	기대값
status	SUBMITTED
txHash	truthy (non-null)
userId	user-001 일치
tokenId	TOKEN-A 일치
amount	100 일치

REQUESTED

DB INSERT



SUBMITTED

txHash 저장

실습 [2] — 역등성 검증

§ 3 실습 · M3 S16

테스트 코드

```
const requestId = randomUUID();

// 동일 requestId로 3회 호출
const r1 = await vasp
  .submitMint({ requestId, ... });
const r2 = await vasp
  .submitMint({ requestId, ... });
const r3 = await vasp
  .submitMint({ requestId, ... });

expect(r1).toBe(r2);
expect(r2).toBe(r3);
```

검증 항목

같은 requestId: r1 === r2 === r3

→ 세 번 모두 동일한 txHash 반환

→ 새로운 TX 생성 없음

```
// 다른 requestId → 다른 txHash
const otherId = randomUUID();
const r4 = await vasp
  .submitMint({ requestId: otherId, ... });

expect(r4).not.toBe(r1);
// r4 = 0xMOCK_0001...(새 순번)
```

submitted Map이 requestId를 key로 캐싱하기 때문에 재호출은 Map.get()만 실행된다

실습 [3] — VASP 실패 → DB FAILED

§ 3 실습 · M3 S16

테스트 코드

```
// 항상 실패하는 VASP mock 주입
const failVasp = {
  submitMint: async () => {
    throw new Error('VASP_UNAVAILABLE');
  }
};

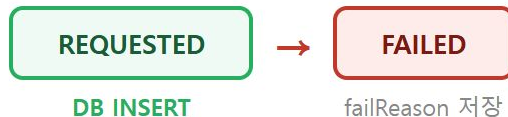
const svc = new TxStateMachineService(
  repo, walletClient, failVasp
);

// submitMintRequest는 throw를 re-throw
await expect(
  svc.submitMintRequest({ ... })
).rejects.toThrow();
```

검증 항목

- > **throw 발생:** submitMintRequest가 예외를 re-throw
- > **DB 레코드 존재:** VASP 실패 후에도 DB 조회 가능
- > **status = FAILED:** REQUESTED → FAILED 전환 완료
- > **failReason 저장:** 'VASP_UNAVAILABLE' 등 에러 메시지 보존

DB-First 원칙의 증거: VASP가 실패하더라도 2단계(DB INSERT)가 먼저 실행되었으므로 레코드가 남는다 — 재처리·감사 추적 모두 가능



실습 [4] — 핸들러 가드

§ 3 실습 · M3 S16

테스트 코드

```
// 이미 CONFIRMED 상태인 요청 생성
const req = await repo.save({
  status: 'CONFIRMED',
  blockNumber: 1000,
  ...
});

// handleMined 재호출 — try-catch 없음
await svc.handleMined(
  req.id,
  9999 // 다른 blockNumber
);

// 검증: throw 없이 통과
const after = await repo
  .findById(req.id);
expect(after.status)
  .toBe('CONFIRMED');
expect(after.blockNumber)
  .toBe(1000); // 변화 없음
```

검증 항목

항목	기대값
예외 발생 여부	throw 없음
status 변화	CONFIRMED 유지
blockNumber 변화	1000 유지 (9999 미적용)
핵심: CONFIRMED 상태에서 handleMined를 다시 받아도 조용히 return — 중복 배달은 오류가 아니다	
at-least-once 환경에서 try-catch 없이 호출해도 서비스가 정상 동작함을 증명하는 테스트	

핵심 정리

S16 역등성 보장: requestId 기반 중복 TX 방어 · M3

#	원칙	구현
1	requestId — UUID, 전 레이어 관통	randomUUID() 발급 → vasp.submitMint({ requestId }) 까지 전달. 재시도 시 동일 id 재사용.
2	VASP 역등성 — submitted Map	같은 requestId → Map.get() 으로 기존 txHash 반환. 새 TX 생성 없음. counter 증가 없음.
3	DB INSERT 먼저	VASP 호출 이전에 REQUESTED 레코드 생성. VASP 실패 → FAILED 기록 남음. 상태 추적 가능.
4	핸들러 가드 — throw 아닌 return	이미 처리된 상태(CONFIRMED 등)에서 재배포 수신 시 return 으로 무시. DLQ 방지. at-least-once 안전.
5	not_found ≠ REVERT	mempool에서 TX 소실(dropped) vs 컨트랙트 실행 중 거절(REVERT)은 다른 케이스. 상태 진단 시 구분 필요.

S16 한 줄 요약: requestId(UUID)를 전 레이어에 관통시키고, DB를 먼저 기록하고, 중복을 오류가 아닌 예상된 케이스로 설계하라

다음 세션 예고

M3 · S17 예정

S16 에서 확인한 것

- › **requestId** — UUID Idempotency Key 전 레이어 관통
- › **VASP 멍등성** — submitted Map으로 중복 TX 차단
- › **DB-First** — VASP 호출 전 상태 기록 선행
- › **핸들러 가드** — 중복 메시지 조용히 무시(return)
- › **at-least-once** — 중복은 오류가 아닌 정상 케이스

S17 에서 다룰 것 (예정)

TX 상태 머신 심화

- FAILED → RETRY 전환 전략
- Exponential Backoff 구현
- 최대 재시도 횟수 제한
- Dead Letter Queue 연동

S16의 멍등성 기반 위에 재시도 전략을 쌓는다 — requestId가 있어야 안전한 재시도가 가능하다

참고 — 용어 정리

M3 S16 · Idempotency 핵심 개념

용어	정의 및 컨텍스트
Idempotency (멱등성)	동일 요청을 여러 번 실행해도 결과가 같은 성질. $f(f(x)) = f(x)$. HTTP PUT-DELETE는 멱등, POST는 비멱등이 기본.
Idempotency Key	요청의 고유 식별자. 재시도 시 동일 key 재사용 → 서버가 중복 판별에 사용. 여기서는 <code>requestId = UUID</code> .
At-Least-Once	메시지 큐 전달 보장 수준 중 하나. 최소 1회 전달 보장하지만 중복 가능. Kafka, RabbitMQ 기본 설정.
Exactly-Once	정확히 1회만 전달. 구현 비용이 매우 높고 성능 저하. 대부분 At-Least-Once + 멱등성으로 대체.
DLQ	Dead Letter Queue. 처리 실패 메시지 보관소. throw로 실패 처리된 메시지가 이동. 운영팀 수동 개입 필요.
DB-First Pattern	외부 시스템 호출 전 DB에 의도(intent)를 먼저 기록하는 패턴. Saga 패턴의 기반. 크래시 안전성 확보.